시스템 프로그래밍

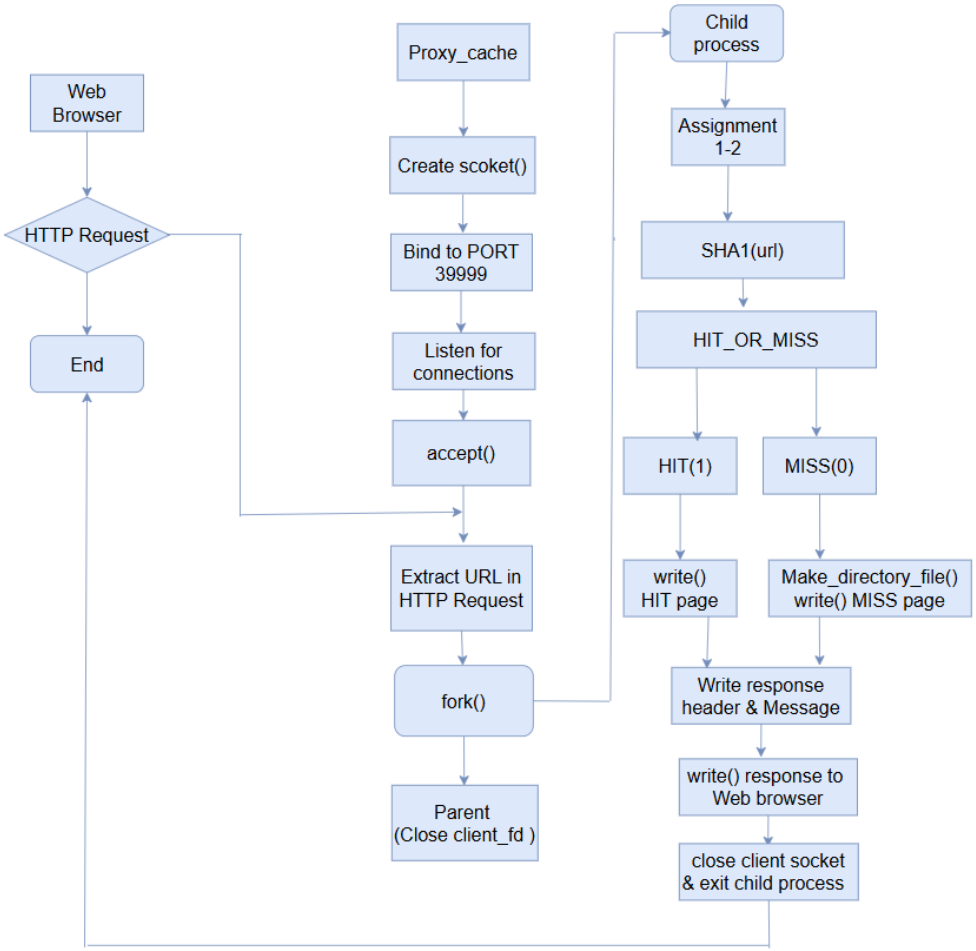
Proxy 2-2

강의 시간	목4 (B)
담당 교수님	최상호 교수님
학번	2021202089
이름	오나균

1. Introduction

이전 과제는 client와 sever간의 요청 응답을 처리하는 것이었는데, 이번 과제는 리눅스 상에서 FireFox를 사용하여, Web browser가 URL을 입력할 경우 이에 대해서, Proxy server가 HTTP request를 받아서 이를 처리하는 것입니다. 이를 처리하는 과정은 proxy_cache.c 파일에 대해서 command 창에서 ./proxy_cache를 입력하여 실행 합니다. 그리고 Web browser에서, URL을 입력하면 (이번 과제에서는 광운대 홈페이지를 사용) Child process는 HTTP request header로부터 URL(host 정보)를 추출하여, 이에 대해서, Assignment 1-2의 기능을 수행하고, Response header와 Response Message를 생성하여 전송한 뒤 Child procees는 종료됩니다.

2. Flow chart



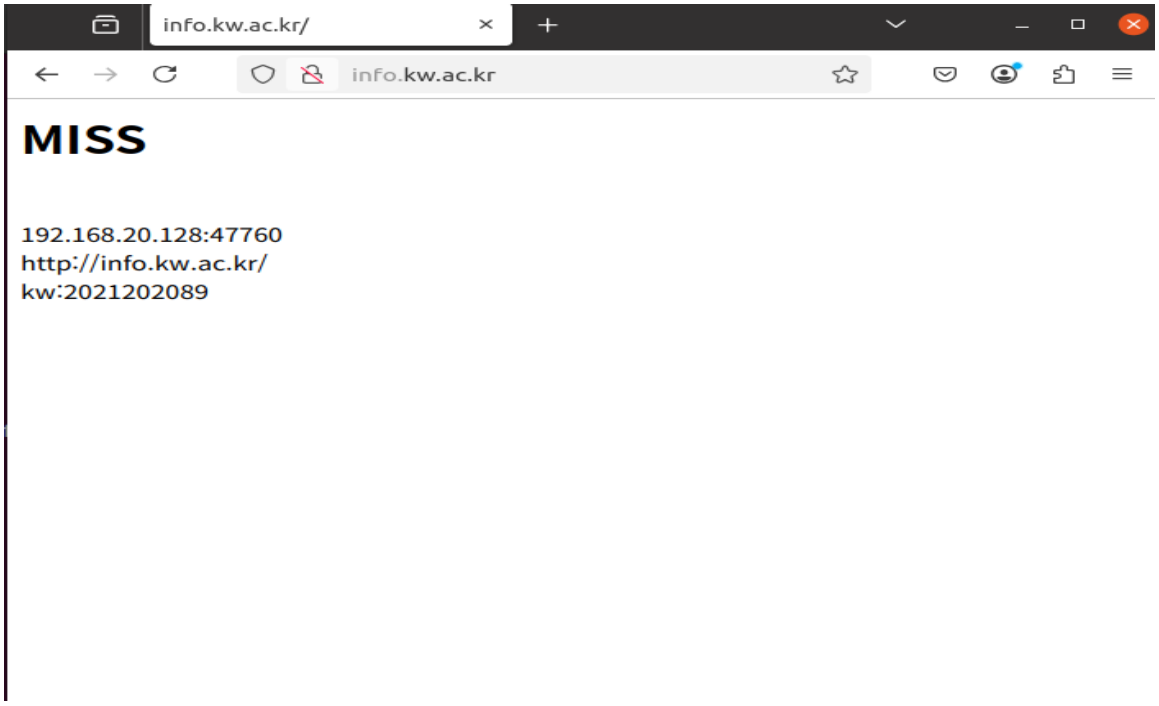
3. Pseudo code

```
int main() {
    Set signal handler to prevent zombie processes
    Create socket
    Bind socket to port
    Start listening for client connections
    while (true):
        Accept client connection
        fork()
        if child process:
            Read HTTP request from client
            Parse method and URL
            if method is GET:
                Hash the URL using SHA1
                Check if the hashed file exists in
                if HIT:
                    Build HTTP response with "HIT"
                else if MISS:
                    Create directories and file based on hashed URL
                    Build HTTP response with "MISS"
                Send HTTP response headers and Message to client
                Close client socket
                Exit child process
        else if parent process:
            Close client socket (parent doesn't handle the connection)
Clean up:
    Close listening socket on termination
```

4. 결과화면

1) MISS일경우

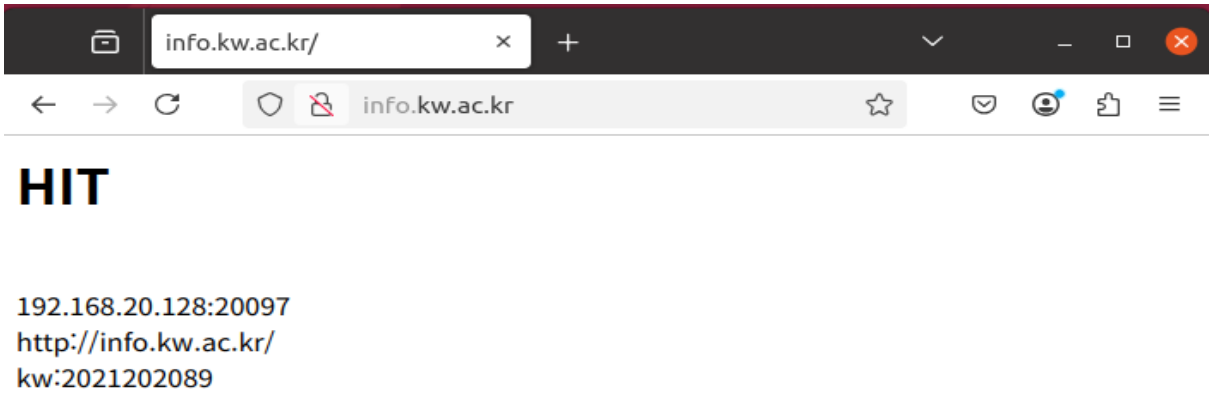
```
[192.168.20.128 : 47760] client was connected
==============================================
Request from [192.168.20.128 : 47760]
GET http://info.kw.ac.kr/ HTTP/1.1
Host: info.kw.ac.kr
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
```



./proxy_sever 명령어를 command 창에서 실행하고 , 최초로 info.kw.ac.kr/ 주소를 입력했을 때 , Cache directory에 해당 url 정보가 없으므로, MISS가 출력됩니다.

2) Hit 일 경우

```
[192.168.20.128 : 20097] client was connected
==============================================
Request from [192.168.20.128 : 20097]
GET http://info.kw.ac.kr/ HTTP/1.1
Host: info.kw.ac.kr
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
```



./proxy_sever 명령어를 command 창에서 실행하고 , 두번째로 info.kw.ac.kr/ 주소를 입력했을 때 , Cache directory에 해당 url 정보가 존재하여, HIT이 출력됩니다.

3) 헷갈리는 출력화면

```
[Miss] ServerPID : 5533 |  - [2025/5/8, 14:15:32]
[Hit] ServerPID : 5538 | da3/9a3ee5e6b4b0d3255bfef95601890afd80709 - [2025/5/8, 14:15:32]
[Hit]
[Miss] ServerPID : 5539 | http://info.kw.ac.kr/ - [2025/5/8, 14:15:33]
[Miss] ServerPID : 5543 | http://info.kw.ac.kr/favicon.ico - [2025/5/8, 14:15:34]
[Hit] ServerPID : 5569 | 4d2/59e95a0f7aae0ff67b0626fb9818a9025a42b - [2025/5/8, 14:15:37]
[Hit]http://info.kw.ac.kr/
kw2021202089@ubuntu:~$ rm -rf ~/logfile/logfile.txt
kw2021202089@ubuntu:~$ tree ~/cache
/home/kw2021202089/cache
├── 4d2
│   └── 59e95a0f7aae0ff67b0626fb9818a9025a42b
├── 6e3
│   └── e3d4ae8eca582787a05d152530ca08ea0a444
└── da3
    └── 9a3ee5e6b4b0d3255bfef95601890afd80709

3 directories, 3 files
```

⇨ 처음에는 로그파일에 info.kw.ac.kr/ 뿐만 아니라 다른 url도 기록되길래, 코드를 잘못 짰나 했는데, 강의 묻고 답하기를 살펴보니까, "HTML 파일 내의 이미지나 리소스가 브라우저에 의해 캐싱되는 것도 오류가 아닙니다" 라고 나와 있어서, 코드를 잘못 짠 것이 아니라 특성이 그런 것임을 알게 되었습니다.

```
kw2021202089@ubuntu:~$ cat ./logfile/logfile.txt
[Miss] ServerPID : 5894 | http://info.kw.ac.kr/ - [2025/5/8, 14:26:10]
[Hit] ServerPID : 5919 | 4d2/59e95a0f7aae0ff67b0626fb9818a9025a42b - [2025/5/8, 14:26:12]
[Hit]http://info.kw.ac.kr/
```

⇨ HTTP request는 모두 받게하되, 최종적으로는 요청한 URL에 대해서만, 로그파일을 기록하게끔 예외처리를 해주었습니다.

5. 고찰

이번 실습과제에서는 리눅스 상에서 Firefox를 사용하여, Web browser가 URL을 입력하여 proxy_server에서 HTTP request를 받아서, 이로부터 host 정보(URL)을 추출하여, 추출한 정보를 토대로, HIT/ MISS 를 판별하여, 웹 브라우저에 Response header 및 Message를 전송하는 것을 구현하는 것 이었다.

이론시간에 배운 내용을 토대로, 프록시 서버를 구현하면서, 이론 시간에는 "아 이런 식으로 사용자와 서버가 연결 되는구나" 싶었던 내용들이, 과제로 직접 수행해보니까, 구체적으로 이해가 되었습니다.

그리고 처음에는 이 리눅스에서 코딩을 하는 것과 이에 대한 Makefile 작성 및 리눅스의 라이브러리를 사용해서, 다양한 함수들과 이에 대해 처리하는 과정이 복잡하게 느껴졌지만, 과제를 일주일마다 연속해서, 수행하다 보니까 확실히 손에 익는 것 같습니다.

또한 이번 과제를 수행하면서 지금까지는 뭔가 코드에서 실행하면 입력을 받고 출력하는 식으로 많이 하였던 것 같은데, 이번 과제에서는 단순히 동작하는 것뿐 아니라, 응답 메시지를 HTML 형식으로 작성하고 이를 웹 브라우저에 메시지로 출력하는 과정도 흥미로웠습니다.

이렇게 웹 브라우저를 직접 사용하여서, 작성한 코드와 직접 상호작용하는 것을 해본 적이 없었는데, 이번에 해보니까, 우리가 보는 웹사이트들이 어떻게 사용자와 상호작용하는지도 알 것 같습니다.

그리고 하면서 궁금하였던 점이, info.kw.ac.kr에만 접속하였는데, command 창에서 하나의 HTTP 요청만 발생하지 않고, 더 많이 발생하는 것이 발견되었습니다. 찾아보니까 웹 브라우저 연결에서는 보안, 성능 최적화등을 위해 백그라운드에서 다양한 요청을 자동으로 발생시킨다는 것 알게 되었습니다. 추가적으로 이 사항을 강의 묻고답하기를 통해 재확인 할 수있었습니다.

6. Reference

HTTP persistent connection - Wikipedia

⇨ 브라우저는 하나의 웹사이트를 요청할 때, 여러 작업을 동시에 또는 순차적으로 한다.